Szoftvertesztelés Dokumentáció

Unit tesztek implementálása meglévő kódbázishoz

Név: Demeter Norbert
Szak: Számtech IV
Csoport: B

2026. március 23.

Tartalomjegyzék

1. A választott unit tesztelő keretrendszer bemutatása **2**
1.1. Motivációk 2
1.2. Telepítés 2

2. A unit teszt kódtár (repository) bemutatása **3**
2.1. Az adatok struktúrája a kódtárban 3

3. A kidolgozott unit tesztek bemutatása **4**
3.1. RelationsManager tesztek 4
3.2. EmployeeManager tesztek 6

4. Kódlefedettség (Code Coverage) **9**

5. CI/CD Automatizálás **10**

1. A választott unit tesztelő keretrendszer bemutatása

1.1. Motivációk

A projekt során a tesztek megírásához a **Pytest** keretrendszert választottam. Bár a Python beépített `unittest` vagy a `nose2` keretrendszerei is megfelelőek lettek volna, a Pytest mellett az alábbi érvek szóltak:

- **Egyszerűség és átláthatóság:** A Pytest lehetővé teszi a kompakt és egyszerű tesztkészletek írását felesleges osztály-hierarchiák és sablonkódok (boilerplate) nélkül.
- **Népszerűség:** Jelenleg ez az egyik legszélesebb körben használt nyílt forráskódú Python tesztelő keretrendszer, hatalmas aktív közösséggel.
- **Sokoldalúság:** Támogatja az egységtesztelést (unit testing), funkcionális tesztelést, valamint az API tesztelést is.
- **Bővíthetőség:** Számos hasznos plugin (pl. pytest-cov, pytest-mock, pytest-randomly) érhető el hozzá, amelyekkel könnyedén növelhető a tesztelés hatékonysága.

1.2. Telepítés

A tesztelési környezet kialakításához egy izolált Python virtuális környezetet hoztam létre, hogy a függőségek ne ütközzenek a globális rendszer csomagjaival.

1. Lépés: A virtuális környezet létrehozása

```
python3 -m venv pytest-env
```

2. Lépés: A virtuális környezet aktiválása Mivel a fejlesztés Windows / Linux környezetben történt, az alábbi parancsot használtam az aktiváláshoz:

```
# Linux/macOS eset n:
source pytest-env/bin/activate

# Windows (PowerShell) eset n:
.\pytest-env\Scripts\activate
```

3. Lépés: A Pytest telepítése A csomagok telepítését a projektben található `requirements.txt` fájl segítségével lehet végre hajtani:

```
pip install requirements.txt
```

2. A unit teszt kódtár (repository) bemutatása

2.1. Az adatok struktúrája a kódtárban

A modern szoftverarchitektúra elveit követve a tesztfájlokat elkülönítettem az alkalmazás üzleti logikáját tartalmazó forráskódtól. Ennek érdekében létrehoztam egy külön `tests/` mappát a projekt gyökérkönyvtárában. A Pytest automatikusan felismeri a `test_` előtaggal kezdődő fájlokat és a bennük lévő tesztfüggvényeket.

A kódtár felépítése az alábbiak szerint alakult:

```
calculator-test/
|-- employee.py
|-- employee_manager.py
|-- relations_manager.py
`-- tests/
    |-- __init__.py
    |-- test_relations_manager.py
    `-- test_employee_manager.py
```

Az elkülönítésnek köszönhetően a projekt átláthatóbb, és a verziókövetés (CI/CD) során is egyszerűbb definiálni a tesztek futtatásának célkönyvtárát.

3. A kidolgozott unit tesztek bemutatása

Ebben a szekcióban a specifikáció által megkövetelt tesztesetek részletes bemutatása következik. Minden tesztesetnél feltüntetem a bemenetet, az elvárt kimenetet, valamint a teszt implementációját.

3.1. RelationsManager tesztek

Ezek a tesztek a munkavállalók adatait és a csapatok felépítését ellenőrzik a `test_relations_manager.p` fájlban.

Test 1: John Doe vezetői státusza és születési ideje

- **Leírás:** Ellenőrzi, hogy létezik-e John Doe nevű csapatvezető a rendszerben, és a születési dátuma pontosan 1970.01.31.
- **Bemenet:** A `RelationsManager` által visszaadott alapértelmezett alkalmazotti lista.
- **Elvárt kimenet:** A keresett dolgozó objektuma nem `None`, az `is_leader()` metódus `True`-t ad vissza, és a születési dátum megfelelő.

```
def test_john_doe_is_leader_and_birthdate(rm):
    john = next((e for e in rm.get_all_employees() if e.first_name
       == "John" and e.last_name == "Doe"), None)

    assert john is not None
    assert rm.is_leader(john) is True
    assert john.birth_date == datetime.date(1970, 1, 31)
```

Test 2: John Doe csapattagjainak ellenőrzése

- **Leírás:** Megvizsgálja, hogy John Doe csapattagjai között szerepel-e Myrta Torkelson és Jettie Lynch.
- **Bemenet:** John Doe alkalmazott objektuma.
- **Elvárt kimenet:** A `get_team_members` metódus által visszaadott listában benne kell lennie a 2-es és 3-as ID-knak, és a lista hossza pontosan 2 kell legyen.

```
def test_john_doe_team_members(rm):
    john = next(e for e in rm.get_all_employees() if e.id == 1)
    team_member_ids = rm.get_team_members(john)

    assert 2 in team_member_ids
    assert 3 in team_member_ids
    assert len(team_member_ids) == 2
```

Test 3: Tomas Andre tagságának kizárása John csapatából

- **Leírás:** Biztosítja, hogy Tomas Andre véletlenül sem tagja John Doe csapatának.

- **Bemenet:** John Doe és Tomas Andre alkalmazott objektumai.
- **Elvárt kimenet:** Tomas Andre ID-ja (5) nem szerepelhet John Doe csapattagjainak listájában.

Test 4: Gretchen Watford alapfizetése

- **Leírás:** Ellenőrzi Gretchen Watford alapfizetésének mértékét.
- **Bemenet:** Gretchen Watford alkalmazott objektuma.
- **Elvárt kimenet:** A `base_salary` attribútum értéke pontosan 4000.

Test 5: Tomas Andre vezetői jogosultságainak hiánya

- **Leírás:** Megbizonyosodik róla, hogy Tomas Andre nem csapatvezető, és ellenőrzi a rendszer viselkedését, ha mégis le kérdeznénk a csapattagjait.
- **Bemenet:** Tomas Andre alkalmazott objektuma.
- **Elvárt kimenet:** Az `is_leader()` függvény `False`-t ad vissza, a `get_team_members()` pedig `None` értékkel tér vissza.

```
def test_tomas_andre_not_leader_and_retrieve_team(rm):
    tomas = next(e for e in rm.get_all_employees() if e.first_name
       == "Tomas" and e.last_name == "Andre")

    assert rm.is_leader(tomas) is False
    assert rm.get_team_members(tomas) is None
```

Test 6: Nem létező dolgozó (Jude Overcash) ellenőrzése

- **Leírás:** Ellenőrzi, hogy Jude Overcash valóban nem szerepel-e az adatbázisban.
- **Bemenet:** Keresés az összes dolgozó listájában Jude Overcash névre.
- **Elvárt kimenet:** A keresés eredménye `None`.

3.2. EmployeeManager tesztek

A fizetésszámítást és e-mail küldést végző logika ellenőrzése a `test_employee_manager.py` fájlban. Mivel a fizetésszámítás az aktuális éven alapul, a tesztek konzisztenciája érdekében a `unittest.mock.patch` segítségével befagyasztottuk az időt a 2018-as évre (a specifikációs példák is ezzel az évvel számoltak).

Test 7: Fizetésszámítás beosztott esetén

- **Leírás:** Egy nem vezető beosztású, 1998.10.10-én felvett dolgozó fizetését számítja ki, akinek az alapfizetése $1000.
- **Bemenet:** Egy mockolt Employee objektum, befagyasztott 2018-as dátum, mockolt RelationsManager.
- **Elvárt kimenet:** $3000 ($1000 alap + 20 év * $100 bónusz).

```
@patch('employee_manager.datetime')
def test_salary_non_leader(mock_datetime):
    mock_datetime.date.today.return_value = datetime.date(2018, 1,
        1)

    mock_rm = MagicMock()
    mock_rm.is_leader.return_value = False

    em = EmployeeManager(mock_rm)
    employee = Employee(id=99, first_name="Test", last_name="User",
                        birth_date=datetime.date(1980, 1, 1),
                        base_salary=1000,
                        hire_date=datetime.date(1998, 10, 10))

    assert em.calculate_salary(employee) == 3000
```

Test 8: Fizetésszámítás csapatvezető esetén

- **Leírás:** Egy 3 fős csapat vezetőjének fizetését ellenőrzi (felvétel: 2008.10.10, alapfizetés: $2000).
- **Bemenet:** Mockolt Employee objektum (vezető), befagyasztott 2018-as dátum.
- **Elvárt kimenet:** $3600 ($2000 alap + 10 év * $100 bónusz + 3 tag * $200 bónusz).

Test 9: Fizetésszámítás és e-mail értesítés

- **Leírás:** Validálja, hogy a fizetésszámítás után a rendszer kiküldi-e a megfelelő szövegű értesítést. Az e-mail küldő szolgáltatást a standard kimenet (print) mockolásával vizsgáltuk.
- **Bemenet:** Az előző tesztben használt csapatvezető objektuma.
- **Elvárt kimenet:** A `print` függvény pontosan egyszer fut le a következő üzenettel: *"Jane Doe your salary: 3600 has been transferred to you."*

```python
@patch('employee_manager.datetime')
@patch('builtins.print')
def test_calculate_salary_and_send_email(mock_print, mock_datetime):
    mock_datetime.date.today.return_value = datetime.date(2018, 1, 1)

    mock_rm = MagicMock()
    mock_rm.is_leader.return_value = True
    mock_rm.get_team_members.return_value = [101, 102, 103]

    em = EmployeeManager(mock_rm)
    leader = Employee(id=100, first_name="Jane", last_name="Doe",
                      birth_date=datetime.date(1985, 1, 1),
                      base_salary=2000,
                      hire_date=datetime.date(2008, 10, 10))

    em.calculate_salary_and_send_email(leader)

    expected_message = "Jane Doe your salary: 3600 has been transferred to you."
    mock_print.assert_called_once_with(expected_message)
```

A tesztek futtatásának eredménye

A tesztek elkészítése után a teljes tesztkészlet futtatása a `pytest -v` parancs kiadásával történt. A `-v` (verbose, azaz bőbeszédű) kapcsoló biztosítja, hogy a konzolos kimeneten részletesen, tesztesetenként (függvényenként) lássuk az eredményeket.

Az alábbi képernyőképen jól látható, hogy mind a `RelationsManager`, mind pedig az `EmployeeManager` osztályokhoz megírt összes teszteset hiba nélkül, sikeresen lefutott (zöld **PASSED** jelzéssel). Ez egyértelműen igazolja, hogy a kódbázis implementált üzleti logikája és az adatok lekérdezése tökéletesen az elvárásoknak és a specifikációnak megfelelően működik.

```
(pytest-env) PS C:\Egyetem\Szoftver_teszteles\vv-proj> pytest -v
================================================== test session starts ==================================================
platform win32 -- Python 3.11.9, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\demet\AppData\Local\Microsoft\WindowsApps\PythonSoftwareF
oundation.Python.3.11_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Egyetem\Szoftver_teszteles\vv-proj
plugins: anyio-4.9.0
collected 12 items

calculator_test.py::test_calc_addition PASSED                                                                          [  8%]
calculator_test.py::test_calc_substraction PASSED                                                                      [ 16%]
calculator_test.py::test_calc_multiply PASSED                                                                          [ 25%]
tests/test_employee_manager.py::test_salary_non_leader PASSED                                                          [ 33%]
tests/test_employee_manager.py::test_salary_leader_with_3_members PASSED                                               [ 41%]
tests/test_employee_manager.py::test_calculate_salary_and_send_email PASSED                                            [ 50%]
tests/test_relations_manager.py::test_john_doe_is_leader_and_birthdate PASSED                                          [ 58%]
tests/test_relations_manager.py::test_john_doe_team_members PASSED                                                     [ 66%]
tests/test_relations_manager.py::test_tomas_andre_not_in_johns_team PASSED                                             [ 75%]
tests/test_relations_manager.py::test_gretchen_watford_salary PASSED                                                   [ 83%]
tests/test_relations_manager.py::test_tomas_andre_not_leader_and_retrieve_team PASSED                                  [ 91%]
tests/test_relations_manager.py::test_jude_overcash_not_in_database PASSED                                             [100%]

================================================== 12 passed in 0.05s ==================================================
```

1. ábra. A teljes tesztkészlet sikeres lefutása a terminálban a `pytest -v` paranccsal

4. Kódlefedettség (Code Coverage)

A megírt tesztek minőségének és alaposságának ellenőrzésére a projekt útmutatójában is javasolt `pytest-cov` bővítményt használtam. Ez a plugin részletes jelentést készít arról, hogy a tesztek futtatása során a forráskód hány százaléka hajtódik végre. A lefedettségi mérést a következő paranccsal végeztem:

```
pytest --cov=.
```

A mérés eredménye alapján a projekt összesített kódlefedettsége (Total Coverage) **95%**-os lett. A megkövetelt üzleti logikát tartalmazó osztályok és metódusok (`RelationsManager`, `Employee`, `calculator.py`) 100%-os lefedettséggel rendelkeznek.

Amint az az alábbi ábrán is látható, az egyetlen kivétel az `employee_manager.py` fájl, amely 78%-os lefedettséget mutat (27 sorból 6 kimaradt). Ennek az az oka, hogy a fájl legalján található egy `if __name__ == '__main__':` blokk, amely csupán az osztályok manuális, konzolos kipróbálását szolgálja. Mivel a unit tesztelés során a modulok importálva lesznek, és nem közvetlenül főprogramként (szkriptként) indulnak el, ez a blokk szándékosan nem hajtódik végre.

```
========================================================= tests coverage =========================================================
========================================================= tests coverage =========================================================
_____________________________________ coverage: platform win32, python 3.11.9-final-0 _____________________________________

Name                              Stmts   Miss  Cover
_____________________________________ coverage: platform win32, python 3.11.9-final-0 _____________________________________

Name                              Stmts   Miss  Cover

Name                              Stmts   Miss  Cover
Name                              Stmts   Miss  Cover
--------------------------------------------------
calculator.py                         6      0   100%
calculator_test.py                   10      0   100%
employee.py                          10      0   100%
employee_manager.py                  27      6    78%
relations_manager.py                 15      0   100%
tests\test_employee_manager.py       33      0   100%
tests\test_relations_manager.py      32      0   100%
--------------------------------------------------
TOTAL                               133      6    95%
=================================================== 12 passed in 0.24s ===================================================
```

2. ábra. A `pytest-cov` által generált kódlefedettségi (coverage) jelentés

5. CI/CD Automatizálás

A modern szoftverfejlesztési elveknek és a projekt opcionális követelményeinek megfelelően egy automatizált CI/CD (Continuous Integration / Continuous Deployment) csővezetéket (pipeline) is integráltam a kódtárba[cite: 146].

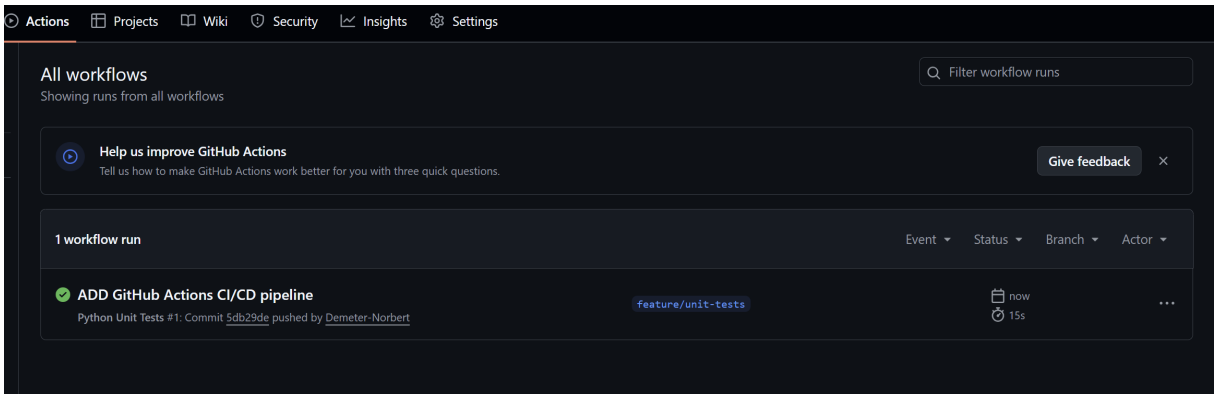
Mivel a verziókövetés a GitHub felületén történik, a feladatra a **GitHub Actions** szolgáltatást választottam. A projekt gyökérkönyvtárában létrehoztam egy `.github/workflows/python-te` konfigurációs fájlt, amely minden új kód feltöltésekor (push), illetve Pull Request nyitásakor automatikusan lefut a felhőben.

A Workflow felépítése

A konfiguráció egy friss Ubuntu Linux környezetet (ubuntu-latest) indít el, majd a következő lépéseket hajtja végre:

1. **Kód letöltése:** A `actions/checkout` segítségével betölti a legfrissebb kódot.
2. **Környezet beállítása:** Felszereli a Python 3.10-es verzióját.
3. **Függőségek telepítése:** A `requirements.txt` alapján feltelepíti a csomagokat, valamint a Pytest keretrendszert és annak bővítményeit.
4. **Tesztek futtatása:** Végrehajtja a `pytest -v -cov=.` parancsot, amellyel ellenőrzi a tesztek sikerességét és a kódlefedettséget.

Ezzel a megoldással garantálható, hogy a fő (main) ágba sosem kerülhet be olyan kód, amely eltöri a már meglévő üzleti logikát, hiszen a GitHub Actions azonnal jelezné a hibát.



3. ábra. A GitHub Actions felületén automatikusan és sikeresen lefutó CI pipeline